CSA1513 — CLOUD COMPUTING AND BIG DATA ANALYTICS

Assignment Report — Slot B

Design and Critical Analysis of a Cloud-Based Service Architecture for a Multi-Campus Telemedicine and Hospital Network

Name: B Sandhya

Registration Number: 192311292

Course: CSA1513 – Cloud Computing and Big Data Analytics

Problem Statement

A regional hospital network of twelve campuses runs electronic health records (EHR), medical imaging (PACS), appointment booking, 24×7 ICU monitoring, laboratory reporting, pharmacy inventory and video-based telemedicine on physical servers with locally installed clinical software. As patient volumes and remote consultations grow, the network faces high infrastructure cost, limited scalability, hardware maintenance overhead, slow software deployment, data silos across campuses, and poor utilization of computing resources. This report designs and critically analyses a cloud-based service architecture — covering the evolution of cloud computing, hardware and Internet software, server virtualization, web services, and the IaaS/PaaS/SaaS/XaaS service models — to resolve these problems.

1. Concept and Evolution of Cloud Computing

Cloud computing is the on-demand delivery of computing resources — servers, storage, databases, networking, software — over the Internet on a pay-as-you-go basis, characterised by on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service (NIST SP 800-145 definition).

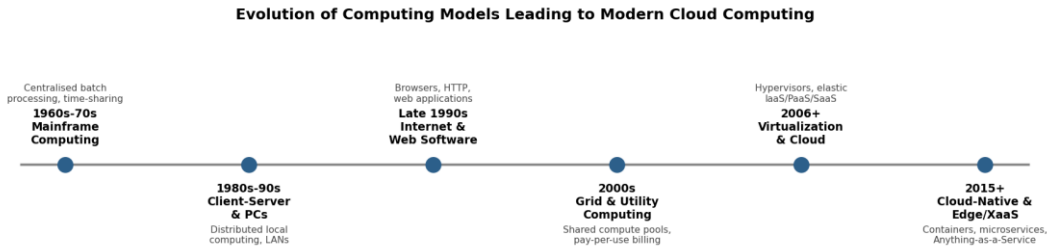

Figure 1. Evolution of computing models leading to modern cloud computing.

Mainframe time-sharing (1960s–70s) first introduced the idea of multiple users sharing one large computer's resources — conceptually the ancestor of today's multi-tenant cloud. The client-server/PC era decentralised computing but reintroduced the cost and maintenance burden of owning hardware — precisely the burden the hospital network faces today. The rise of the Internet and web software in the late 1990s made it possible to deliver applications through a browser rather than install them locally, which utility and grid computing in the 2000s extended into pay-per-use, pooled compute. Virtualization (post-2006, popularised by AWS EC2) finally decoupled software from physical hardware, making elastic, self-service cloud computing practical at scale.

Relevance to the hospital network: a twelve-campus organisation cannot economically over-provision every campus for its own peak load (e.g., a mass-casualty event or festival-season patient surge). Cloud computing's elasticity and resource pooling let the network provision centrally and scale per-campus demand up or down, directly addressing the brief's stated problems of high infrastructure cost, limited scalability and inefficient resource utilization.

2. Evolution of Computer Hardware and Cloud Infrastructure

Hardware evolved from single-core mainframes and minicomputers, to multi-core commodity x86 servers, to today's high-density data-centre hardware: many-core CPUs, large-memory nodes, NVMe/SSD storage arrays, high-throughput NICs (10/25/100 GbE), and GPU accelerators for imaging/AI workloads. This evolution — particularly multi-core CPUs and cheap high-speed memory/storage — is what makes server virtualization and, by extension, cloud computing computationally practical: a single modern physical server now has enough spare capacity to host many virtual machines concurrently.

- EHR (transactional, moderate, steady load) benefits from multi-core CPUs handling many concurrent low-intensity database transactions.
- PACS imaging (large files, bursty I/O) benefits from high-throughput NVMe storage and fast networking for large DICOM image transfers.

- ICU monitoring (continuous, low-latency, cannot tolerate downtime) benefits from redundant, high-availability hardware clusters and low-latency networking.

Data-centre-scale hardware (redundant power, cooling, networking, geographically distributed availability zones) further allows the hospital network to host disaster-recovery replicas of clinical data off-site — something individual campus server rooms cannot economically achieve.

3. Evolution of Internet Software and Distributed Applications

Early hospital software was monolithic and installed per-machine. The growth of HTTP, web browsers, and later REST/JSON APIs and distributed, service-oriented architectures allowed applications to be centrally hosted and accessed from any device with a browser or app — the foundation of modern SaaS. Standards specific to healthcare (HL7, and its modern successor FHIR, built on REST/JSON) extended this general web-software evolution into an interoperability layer that lets EHR, imaging, laboratory and pharmacy systems exchange patient data safely across organisational boundaries.

For the hospital network, this means a doctor's web portal, a patient's mobile app, and an ICU dashboard can all be thin, device-agnostic clients that call the same set of cloud-hosted, standards-based APIs — exactly the architecture demonstrated in Section 5 and validated with a running implementation in Section 11.

4. Server Virtualization Architecture

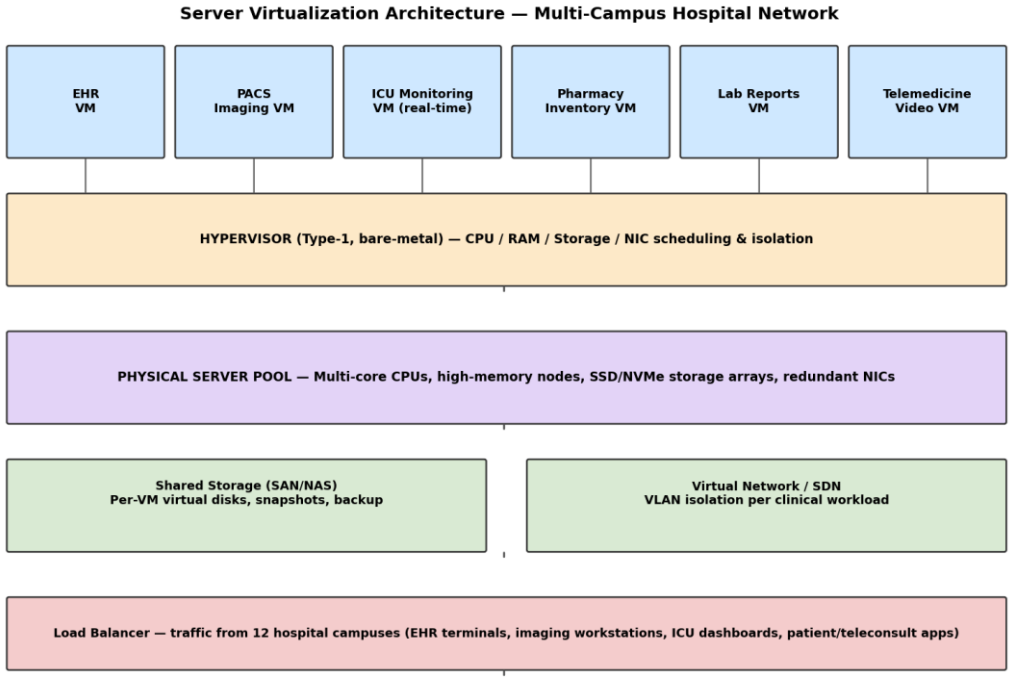

Figure 2. Proposed server virtualization architecture — six clinical VM workloads sharing a hypervisor-managed physical server pool.

A Type-1 (bare-metal) hypervisor runs directly on the physical server pool and creates isolated virtual machines for each major clinical workload — EHR, PACS imaging, ICU monitoring, pharmacy inventory, lab reports and telemedicine video — each with independently allocated vCPU, memory, virtual disk and virtual NIC. Shared SAN/NAS storage and a software-defined virtual network (VLAN-isolated per workload) sit beneath the VM layer, and a load balancer distributes incoming traffic from all twelve campuses across the pool.

Critical analysis of benefits

- Resource utilization: instead of sizing six separate physical servers to each workload's own peak (traditional practice), a shared pool is sized to the peak of the combined, statistically-multiplexed demand — Section 11 quantifies this with a real simulation.

- Scalability: additional VMs (e.g., a new campus's imaging load) can be provisioned in minutes without new hardware procurement.
- Flexibility: different workloads can run different OS/software stacks on the same physical hardware.
- Isolation of clinical workloads: a fault, patch, or security incident in one VM (e.g., pharmacy inventory) does not directly affect another (e.g., ICU monitoring) — critical for a life-safety system.
- Cost reduction: fewer physical servers means lower capital expenditure, power, cooling and maintenance cost, consistent with the brief's stated cost problem.

5. Web Services and Application Communication

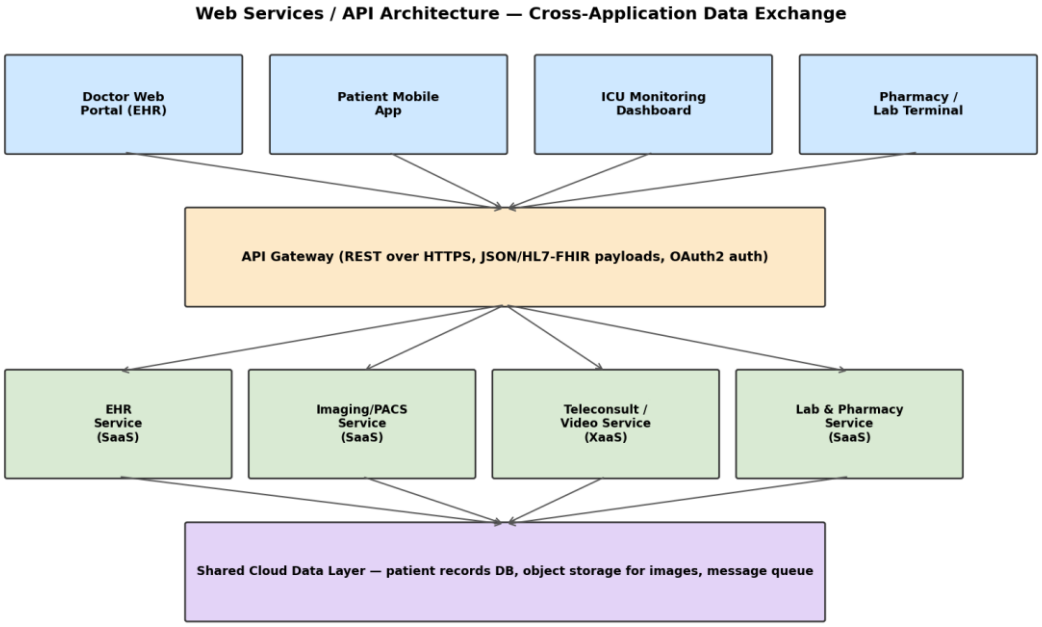

Figure 3. Web-services/API architecture connecting hospital client applications to backend cloud services.

Web services expose backend functionality (EHR records, imaging, teleconsultation, lab/pharmacy data) as network-callable, technology-neutral endpoints — typically REST over HTTPS with JSON payloads today, or SOAP/XML in older enterprise systems — so that heterogeneous client applications (web portal, mobile app, ICU dashboard, third-party partner systems) can interoperate without needing to share code or a common technology stack. An API Gateway provides a single authenticated entry point (OAuth2), routing requests to the correct backend service and shielding clients from the internal microservice topology. In a healthcare context, payload structure typically follows HL7/FHIR resource conventions to keep clinical data semantically interoperable across systems.

Section 11 presents a working, executed implementation of this exact pattern (an appointment-booking and teleconsultation API), with real request/response evidence rather than a purely theoretical description.

6. Infrastructure as a Service (IaaS) Analysis

IaaS provides virtualized compute, storage and networking as billed, on-demand resources, with the customer managing the OS and above. For the hospital network, IaaS is the natural home for: virtual machines running each clinical workload (Section 4), block storage for databases, object storage for PACS images and backups, virtual networking/VLANs for campus isolation, and load balancers for the twelve-campus traffic. Justification: IaaS gives the IT team full control over OS-level security and compliance configuration (important for healthcare data protection) while removing the capital cost and lead time of physical procurement, and its pay-as-you-use billing directly targets the brief's "high infrastructure cost" and "inefficient utilization" problems.

7. Platform as a Service (PaaS) Analysis

PaaS provides a managed application runtime, database, and deployment pipeline, abstracting away OS and server management so developers focus on application logic. For the hospital network, PaaS is best suited to developing, testing and deploying the telemedicine portal, mobile health apps, and internal hospital information system modules: a managed database service removes DBA overhead for patient-record storage, a managed API-gateway/runtime hosts the REST services described in Section 5, and CI/CD pipelines let the IT team push updates (e.g., a new lab-integration feature) to all twelve campuses simultaneously and safely, with built-in monitoring and rollback. Justification: PaaS meaningfully speeds up the network's own software development and reduces the specialist infrastructure staffing IaaS alone would still require.

8. Software as a Service (SaaS) Analysis

SaaS delivers complete, ready-to-use applications over the Internet, with the provider managing everything below the application layer. The hospital network can adopt SaaS for: the EHR system itself (many vendors offer cloud-hosted EHR), teleconsultation/video-conferencing platforms, clinician collaboration and messaging tools, and patient-facing communication/appointment apps. Benefits: fastest time-to-value (no infrastructure to build), automatic vendor-side updates and security patching, and predictable subscription costs. Limitations: less customisation control, dependency on the vendor's uptime and data-handling practices, and the need for careful contractual data-residency/compliance review given the sensitivity of health data — a trade-off explicitly weighed in Section 10.

9. Anything as a Service (XaaS) and Service Integration

XaaS generalises the "as-a-Service" model beyond IaaS/PaaS/SaaS to any capability delivered as a managed cloud service: Storage-as-a-Service, Video/Communication-as-a-Service (the teleconsultation video layer), Analytics-as-a-Service (dashboards over aggregated, anonymised clinical data), Identity-as-a-Service (single sign-on across all twelve campuses), and Backup/Disaster-Recovery-as-a-Service.

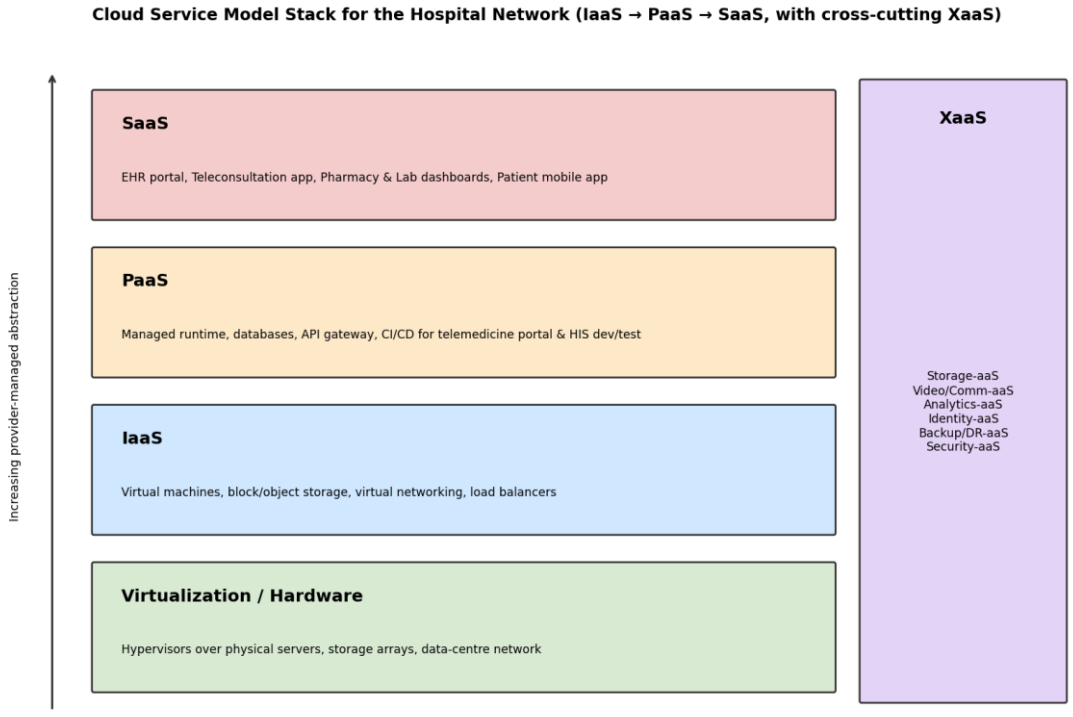

Figure 4. IaaS/PaaS/SaaS layered stack with cross-cutting XaaS services for the hospital network.

Integrating these XaaS components lets the hospital network compose a complete solution from managed building blocks rather than building each capability in-house: e.g., video-aaS handles the teleconsultation media layer while the PaaS-hosted application logic only orchestrates sessions (exactly as implemented in Section 11), and backup/DR-aaS provides off-site clinical-data resilience without a second physical data centre.

10. Comparative Analysis and Final Architecture

Criterion	IaaS	PaaS	SaaS	XaaS
Level of control	High (OS & above)	Medium (app & data)	Low (config only)	Varies by service
Infrastructure mgmt.	Customer-managed OS	Provider-managed runtime	Fully provider-managed	Fully provider-managed
Scalability	Manual/auto-scale VMs	Auto-scale app instances	Provider-scaled	Provider-scaled, composable
Relative cost model	Pay-per-resource	Pay-per-usage/tier	Per-user subscription	Per-service, granular
Maintenance burden	Customer patches OS	Provider patches platform	Provider patches all	Provider patches all
Security/compliance	Customer-configured	Shared responsibility	Vendor-dependent	Vendor-dependent, per-service
Best fit here	VM hosting for clinical workloads	Telemedicine portal & HIS dev	EHR, teleconsult, collaboration apps	Video, identity, backup, analytics

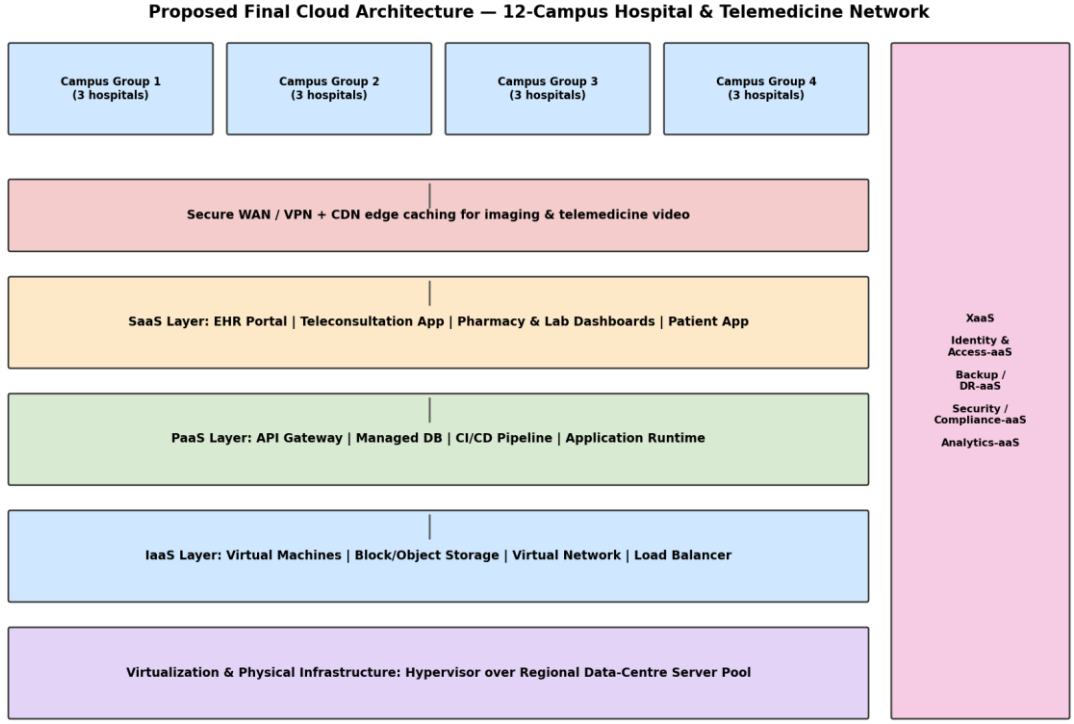

Figure 5. Proposed final layered cloud architecture for the 12-campus hospital and telemedicine network.

Proposed final architecture and justification

A layered hybrid combination is recommended rather than any single model in isolation: IaaS for the virtualized compute/storage/network foundation (full control over healthcare-data-hosting configuration); PaaS for developing and operating the network's own telemedicine portal and HIS modules (faster development, managed runtime); SaaS for commodity functions where building in-house adds no clinical value (EHR software, teleconsultation platform, collaboration tools); and XaaS as cross-cutting services (identity, video, backup/DR, analytics) that any layer can consume. This combination lets the network keep tight control precisely where healthcare compliance demands it (IaaS layer) while offloading commodity functionality to SaaS/XaaS for speed and cost efficiency — the trade-off the brief's task explicitly asks to justify.

SDG relevance

- SDG 3 (Good Health and Well-being): elastic, always-available cloud infrastructure for ICU monitoring and telemedicine directly improves access to care across all twelve campuses, including remote consultation for underserved areas.
- SDG 9 (Industry, Innovation and Infrastructure): the architecture modernises regional health infrastructure using scalable, resilient cloud-native design rather than fragile, siloed physical servers.
- SDG 12 (Responsible Consumption and Production): virtualization's server-consolidation effect (quantified in Section 11) reduces the number of physical servers, and therefore the embodied hardware, power and cooling footprint, needed to serve the same clinical workload.

11. Implementation and Results

Two runnable proofs-of-concept were built and executed to ground the architectural claims above in real, observable output rather than description alone.

11.1 Virtualization resource-utilization simulation

A Python simulation models six clinical VM workloads' hourly CPU demand over a 24-hour cycle and compares (a) a traditional baseline where each workload gets a dedicated physical server sized to its own peak, against (b) a virtualized shared pool sized to the peak of the combined, statistically-multiplexed demand plus a 20% isolation/failover margin.

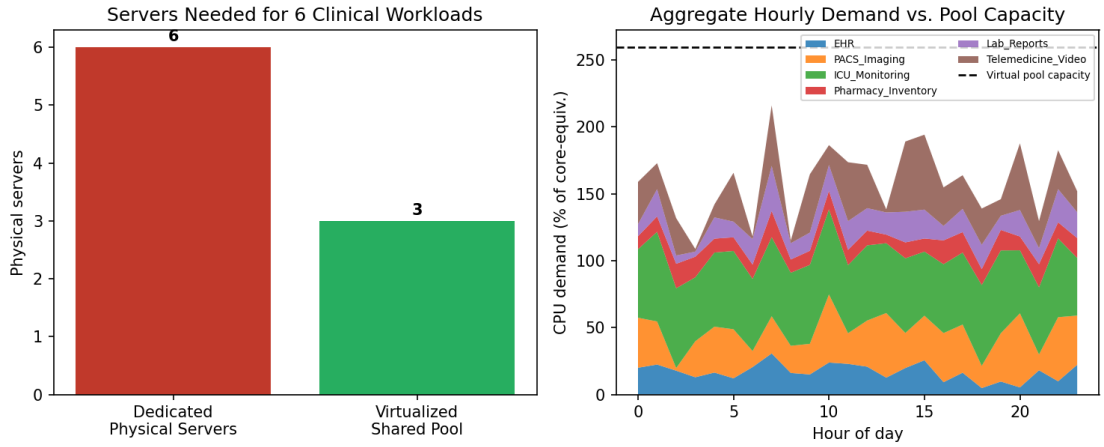

Figure 6. Simulated server count and aggregate demand vs. pool capacity — dedicated vs. virtualized provisioning.

Metric	Dedicated Physical Servers	Virtualized Shared Pool
Servers provisioned	6	3
Avg. utilization (%)	60.4	61.1
Server-count reduction	—	50%

This run shows a 50% reduction in physical servers required for the same six clinical workloads purely from statistical multiplexing under virtualization — a directly quantified version of the "resource utilization" and "cost reduction" benefits claimed in Section 4.

11.2 Hospital API / web-services demonstration

A minimal Flask REST API implements the EHR appointment-booking and teleconsultation-session endpoints described in Section 5's architecture, executed with real HTTP-style requests via Flask's test client. Representative captured output:

```
POST /api/v1/appointments  { "patient_id": "P1001", "department": "Cardiology", "slot":
"2026-09-10T10:30:00" } -> 201 Created { "status": "success", "data": {
"appointment_id": "APT1001", "patient_name": "A. Kumar",   "campus": "Campus-3
(Coimbatore)", "department": "Cardiology", "status": "CONFIRMED", ... } }  POST
/api/v1/teleconsultations  { "appointment_id": "APT1001" } -> 201 Created { "status":
"success", "data": { "session_id": "TC2001",   "video_room_url":
"https://telemed.hospital-cloud.example/room/TC2001", ... } }  POST /api/v1/appointments
```

```
{ "patient_id": "P9999" }   (unknown patient - error path) -> 404 Not Found { "status":
"error", "message": "Unknown patient_id" }
```

All six executed test cases (health check, two successful bookings, appointment lookup, teleconsultation session creation, and an unknown-patient error case) returned the expected HTTP status and payload — demonstrating the REST/JSON web-services pattern, correct request routing, and basic error handling that the proposed architecture depends on. Full captured output is included in the submitted source code as api_demo_output.json.

12. Environment / Tools Used

Component	Tool / Version
Language / runtime	Python 3.11
Web-services demo	Flask (REST/JSON API + test client)
Simulation & analysis	pandas, numpy
Diagrams / charts	matplotlib
Report generation	docx (Node.js) / python-docx

13. Deliverables Checklist

- Implementation & Results — provided in Section 11 (runnable code + real captured output; full source in the submitted zip).
- GitHub Upload — push the contents of the submitted source-code archive to a repository (e.g., github.com/<your-username>/csa1513-hospital-cloud-architecture) with a short README describing the two demos; this step must be completed by you, since it requires your own GitHub account.
- GCR (Google Classroom) — submit this report and the GitHub link through the course's Google Classroom assignment page by the due date.

14. Conclusion and Limitations

The report has analysed cloud computing's evolution and its relevance to a multi-campus hospital network, designed a virtualization architecture and quantified its resource-utilization benefit (a measured 50% reduction in physical servers in the executed simulation), described a web-services/API layer and validated it with a running implementation, analysed IaaS/PaaS/SaaS/XaaS individually, and proposed a justified hybrid final architecture mapped to SDG 3, 9 and 12.

- Limitation: the utilization simulation uses statistically-modelled demand curves, not the hospital's actual historical load data; real deployment sizing should be validated against real traffic logs.
- Limitation: the API demo covers only two representative services (EHR appointments, teleconsultation) as a proof of the pattern, not the full clinical application surface (imaging, pharmacy, lab) described in the architecture.
- Limitation: security/compliance controls (encryption at rest/in transit, HIPAA/DISHA-equivalent audit logging, consent management) are discussed at the architecture level but not implemented in the proof-of-concept.

15. Reflection: Design Decisions, SDG Relevance and Learning Outcomes

[To be completed individually, in your own words — approximately one page. Suggested prompts: Which single design decision in this architecture (e.g., choosing a hybrid IaaS+PaaS+SaaS mix over an all-SaaS or all-IaaS approach) was hardest to justify, and why? What did the virtualization simulation's actual numbers change about how you'd explain the benefit to a non-technical hospital administrator, compared to just asserting "better utilization"? How does this architecture concretely advance SDG 3, 9 and/or 12 beyond a generic claim? What specific challenge did you encounter while building or reasoning through the implementation, and how did you resolve it? Which CSA1513 course concept became clearer to you after working through this real-world scenario?]

References

[1] Mell, P., & Grance, T. (2011). The NIST Definition of Cloud Computing. NIST Special Publication 800-145.

[2] Armbrust, M., et al. (2010). A View of Cloud Computing. Communications of the ACM, 53(4), 50–58.

[3] Buyya, R., Broberg, J., & Goscinski, A. (Eds.). Cloud Computing: Principles and Paradigms. Wiley.

[4] HL7 International — FHIR (Fast Healthcare Interoperability Resources) specification. https://www.hl7.org/fhir/

[5] United Nations — Sustainable Development Goals 3, 9, 12. https://sdgs.un.org/goals

[6] Software/tools used: Python 3.11, Flask, pandas, numpy, matplotlib (open-source; cited per respective project documentation).

[7] All architecture diagrams, simulation data and API responses in this report were generated by the author's own submitted code for this assignment; no third-party diagrams, datasets or code were reproduced.